

What is Python?

Warm Up

```
# Traditional greeting
print("Hello, Python!")

# Simple calculation
x = 10
y = 5
print(x + y)
```

Python Syntax Example

Objectives



Python Structure

Understand basic coding structure and syntax within the Python environment.



Data Types

Understand the fundamental differences and use cases for various data types.



Flow Control

Become familiar with the logic behind loops and conditional statements.



Functions & Lists

Become familiar with creating functions and managing data in lists.

Mastering these core components is essential for effective data analysis in Python.

Why Python for Data Science?



The common question for beginners:

"Why do we use Python over a different language? Why not Java or C?"

To answer this question, we must examine the fundamental nature of Python itself.

What is Python?

Python is a powerful, easy-to-learn programming language that's used for a wide range of tasks.

It's often called a **"general-purpose"** language because it can be used to create everything from websites and apps to scientific experiments and games.

Python is especially popular because it's **beginner-friendly**, with simple and readable code that's easy to understand—even if you're just starting out.

```
# Python is readable
print("Hello, World!")

# Simple syntax
total = 10 + 5
print(total)
```



Versatile & Friendly

Why is Python Popular?

Python has become the go-to language for data science for a few key reasons:



Libraries for Everything

Pre-built functions like Pandas and NumPy make data work fast and efficient.



Community Support

A massive community means there is almost always an answer online if you get stuck.



Flexibility

Works perfectly for data cleaning, machine learning, and creating dashboards.



Readability

Simple and clean syntax lets you focus on solving problems, not tricky code.

In this lesson, we will learn Python basics and apply them to a word usage project.

Activity: Python Basics

Now that you've set up your environment, let's dive into essential concepts. Remember: **practice makes progress**. True mastery comes from being able to explain and execute, not just passive reading.



Hands-On Learning

Follow along with each section. Enter and run the code yourself to truly internalize the logic.



Rule of Thumb

Don't copy and paste. Type the code manually to build muscle memory and learn the syntax for different Python basics.

Variables & Data Types

A variable is like a container for storing information. They make it easy to reuse and update information in your code.

String (text)

```
name = "Alex"
```

Integer (whole number)

```
age = 16
```

Float (decimal number)

```
height = 5.9
```

Boolean (True/False)

```
is_student = True
```

Data Type Interaction Rules

Datatypes follow certain rules for how they interact with other data types. Try out the following code to see what happens when rules are broken:

INPUT

Python Code

```
x = 5 # int
y = "2" # str
result = x + y
```

ERROR

The Result: An Error

TypeError: unsupported operand type(s) for +: 'int' and 'str'

Python cannot automatically add a number to a piece of text.

Data Type Compatibility

In Python, data types determine how values behave and interact. Some types can be combined ("operated on") together, while others cannot. Understanding compatibility is key to handling mismatches.



Try It Yourself: Avoiding Errors

Notice: If we removed the " " around the 2 we would not get an error message. Try this on your own to see that it works. You should not get any message after running that code.

```
# Example Fix: x = 5 + 2
```

Common Python Data Types

There are 15 different data types in Python, but these are the most common ones encountered in everyday programming:

str (string)

Used for text content.

int (integer)

Used for whole numbers.

float (decimal)

Used for numbers with decimals.

bool (boolean)

Used for True or False statements.



Interaction Rule: float + int = float

Adding a float and an int together always results in a float. Other combinations require explicit type changes.

Note: Variables store information from users, functions, or operations. See more details on the next page.

2. Input and Output

Certain functions allow for users to enter information, which then can be used in other functions.

Displaying Information

OUTPUT

Use the `print()` function:

```
print("Hello, world!")
```

Asking for Input

INPUT

Use the `input()` function:

```
name = input("What is your name? ")  
print("Nice to meet you, " + name + "!")
```



Context: Notice we took a user's input, stored it in a variable 'name', and then used that variable in our print function.

This is a common interaction. Think of websites that greet you by name—this is the logic running in the background.

3. Conditional Statements

Flow control statements enable programs to make decisions, repeat actions, or skip sections of code based on conditions you set.

```
# If/Else Logic
```

```
age = 16
```

```
if age >= 18:
```

```
    print("You can vote!")
```

```
else:
```

```
    print("You're too young to  
vote.")
```

PYTHON

How it Works

- The program checks if `age >= 18` is true.
- **Indentation** (spaces) is mandatory in Python to define code blocks.
- If the statement is false, it executes the code under the `else` statement.



These are not the only conditional statements; we will explore more complex phrases like **elif** in the future.

4. Loops

Loops are flow control statements that allow you to repeat an action a certain amount of times depending on whether it is a 'for' loop or a 'while' loop.

```
# For Loop Example
```

PYTHON

```
for i in range(5):  
    print("This is loop number", i)
```

How it Works

Iterates over sequences like lists, tuples, or ranges.

Executes a block of code multiple times.

The **syntax** involves a keyword, an iterator, and a collection.



Notice the structure: A 'for' loop in Python is used to iterate over a variety of datatypes like sequences of letters or numbers.

Basic Syntax of a for Loop

```
# Python For Loop Syntax
```

PYTHON

```
for variable in sequence:  
    # Code to execute in each  
    iteration
```

Component Breakdown

Variable: Often 'i' or 'x', but can be any name representing an item in the collection.

Sequence: A collection of elements (list, string, range) to iterate through.

Indentation: Mandatory to define the block of code belonging to the loop function.



By tradition, many coders use **i** or **x** as variables, but descriptive names are often better. The **sequence** can be any iterable, such as a list of numbers, a string of letters, or a range.

Iterating with the range() Function

The **range(n)** function generates a sequence of integers from 0 up to, but not including, **n**.

PYTHON

```
# Using range(5)
for i in range(5):
    print(i)

# Using a list of words
for item in ['tomato', 'apple']:
    print(item)
```

Key Takeaways

- **range(5)** produces the sequence: [0, 1, 2, 3, 4].
- In each iteration, the variable is replaced by the next value in the sequence.
- Sequences don't have to be numbers; they can be lists of strings like ['tomato', 'apple'].



Replacing a numerical range with a list of words will update the loop variable with those specific words in each cycle.

Example 1: Looping Through a List

Python List Iteration

```
fruits = ["apple", "banana", "cherry"]
```

```
for fruit in fruits:  
    print(fruit)
```

Output:

```
apple  
banana  
cherry
```

PYTHON

Explanation

- The **loop** visits each item in the `fruits` list sequentially.
- The **fruit** variable acts as a placeholder for the current item.
- Python uses descriptive names (`fruit` vs `fruits`) to make code readable.



In this example, the variable **fruit** is updated in each cycle with a value from the sequence **fruits** until all items have been processed.

Example 2: Looping Through a String

```
# String Iteration
```

```
word = "Python"
```

```
for letter in word:  
    print(letter)
```

```
# Output:
```

```
P  
y  
t  
h  
o  
n
```

PYTHON

Explanation

- The loop iterates over each character in the string "Python".

Key Takeaways

- A for loop iterates through a sequence.
- The loop variable takes on each value from the sequence.
- `range(n)` is useful for repeating actions a specific number of times.



Just like with lists, Python's for loop treats a string as a sequence of characters, making it easy to process text character-by-character.

Understanding While Loops

A **while loop** executes code repeatedly **as long as a condition remains True**, continuing until it is no longer met.

PYTHON

```
# Example 1: Basic While Loop  
count = 0  
while count < 5:  
    print("Count is", count)  
    count += 1
```

Output

```
Count is: 0  
Count is: 1  
Count is: 2  
Count is: 3  
Count is: 4
```



Unlike for loops that iterate over a fixed sequence, while loops are dynamic and rely on logical conditions to terminate.

How the While Loop Works

In the previous example, we initialize the **count** variable to zero.

The loop executes **as long as count is less than 5**. Inside the loop:

- The current **count** value is printed.
- The operator **+=** adds 1 to the count, updating it for the next iteration.
- This cycle repeats until **count** reaches 5, at which point the condition becomes False and the loop ends.



The increment step (**count += 1**) is critical; without it, the loop would never end, creating an "infinite loop."

While Loop for User Input

A **while loop** is ideal for scenarios where the number of iterations is unknown beforehand, such as waiting for specific **user input**.

```
# Interactive Password Check
```

PYTHON

```
password = ""  
while password != "secret":  
    password = input("Enter password: ")  
print("Access granted!")
```

How it Works

- **Initialization:** The variable is set to an empty string to ensure the loop starts.
- **Condition:** The operator `!=` checks for inequality.
- **Termination:** The loop ends immediately once the condition becomes **False** (input matches "secret").



Unlike **for** loops which iterate over fixed sets, **while** loops provide the flexibility needed for interactive or event-driven logic.

Avoiding Infinite Loops

Be careful! If the loop condition never becomes **False**, the loop runs forever. Research your system's shortcuts to manually terminate stuck processes.

Example 3: Loop Safety

PYTHON

```
x = 10
while x > 0:
    print("Counting down:", x)
    x -= 1 # Critical update
```

Why it's Safe

- **Decrementing:** The `x -= 1` line ensures `x` eventually reaches 0.
- **Termination:** Once `x` is not `> 0`, the condition becomes False.
- **Safety:** Without this update, the value of `x` stays at 10 forever.



Always ensure your loop logic includes a **terminating condition** that can be reached during standard execution.

Using break to Stop a While Loop

You can **manually exit** a while loop using the **break** statement, providing precise control over loop termination.

PYTHON

```
# Example 4: Break Statement

number = 1
while number < 10:
    print(number)
    if number == 5:
        print("Stopping the loop.")
        break # Stops early
    number += 1
```

Output

1
2
3
4
5

Stopping the loop.



Explanation: The loop condition is `number < 10`, but it **stops early** at 5 because the `break` statement is triggered.

Using continue to Skip an Iteration

The **continue** statement skips the rest of the current loop iteration and immediately moves to the next one.

PYTHON

```
# Example 5: Continue Statement  
x = 0  
while x < 5:  
    x += 1  
    if x == 3:  
        continue # Skips printing 3  
    print(x)
```

Output

1
2
3
4
5



Explanation: When `x == 3`, the `continue` statement skips that iteration. The `==` operator checks for equality (Truth statement).

5. Functions

Functions are **reusable blocks of code** that perform specific tasks, allowing you to organize and repeat logic efficiently.

Defining and Calling a Function

PYTHON

```
def greet(name):  
    print("Hello, " + name + "!")  
greet("Alex") # Execution
```

How it Works

- **Definition:** We name the function **greet** with an input variable (parameter) named **name**.
- **Input:** Passing **"Alex"** sets the value of name for that specific call.
- **Logic:** The function uses the variable inside its block to perform its task.



Functions are fundamental in programming. We will dive deeper into complex structures in **Unit 2**.

6. Lists

A **list** is a collection of items that allows you to store multiple values in a single variable.

Creating and Accessing a List

PYTHON

```
fruits = ["apple", "banana", "cherry"]  
print(fruits[0]) # Prints "apple"
```

Key Features

- **Zero-Indexed:** The first item is 0, the second is 1, and so on.
- **Mutable:** You can add, remove, or change items easily.
- **Syntax:** Lists are defined using square brackets [].



Remember: **fruits[0]** refers to "apple". In Python, counting always starts at zero, not one.

7. Comments

Use **#** to add comments to your code. Comments help explain what your code does but are **ignored** when the code runs.

```
# Python Code with Comments
# This prints a greeting
print("Hello, world!")
```

PYTHON

Best Practices

-

Notebooks: We often use markdown for long explanations in Jupyter.

-

Inline: Use `"#"` for quick explanations directly inside code blocks.

-

Clarity: Keep comments concise and helpful for your future self.



Practice makes perfect. Documenting your logic is as important as writing the code itself.

Coding Practice

Now that you've learned some basic concepts, try practicing them in your **Jupyter Notebook**:

Task List

- Create a variable and print its value.
- Write a loop that prints numbers 1-10.
- Create a greeting function with name input.



Experiment and learn. Don't be afraid to make mistakes—they are a critical part of the coding process.

Module Summary

Congratulations! You've mastered the **fundamental building blocks** of Python programming.

01. Functions

- Reusable code blocks
- Input parameters
- Logic organization

02. Lists

- Zero-indexed storage
- Mutable values
- Square bracket [] syntax

03. Comments + Flow Control

- The "#" symbol usage
- Explaining code intent
- Ignored by compiler
- For/While Loops



Ready for Unit 2. Apply these concepts in your final project to solidify your understanding of Python basics.